

A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js

Mateusz Miotk^{a,*}

^a*Faculty of Mathematics, Physics and Informatics, University of Gdańsk, Wita Stwosza 57, 80-308, Gdańsk, Poland*

Abstract

Context: Benchmarks of relational database access layers inform technology-selection decisions, yet can time implementations that returned different results, or report latency at unequal proximity to saturation. Whether the compared configurations are comparable is rarely established. **Objectives:** We propose an executable protocol for deciding when configured implementations are comparable for a stated estimand, and what their measurements support, then exercise it in one empirical instantiation rather than validating it independently. **Methods:** The protocol requires task-derived read checks, cross-implementation equivalence, post-write state validation, a predeclared configuration-selection policy, and separate capacity and latency operating points. An open-source Express.js experiment compares nine policy-selected configurations and two tuned native references across PostgreSQL, MySQL, and five access patterns in a 25-run randomized-block campaign, with a reduced second same-host campaign testing rank stability and a single-rater audit of nine external benchmarks. **Results:** The widest spread is the tested deep fetch, $7.01\times$ on PostgreSQL and $5.12\times$ on MySQL; forcing common SQL through raw facilities in a separate ten-run campaign leaves $1.71\times$ and $2.03\times$, a compound contrast isolating no single mechanism. High-load and matched-utilization p99 are different quantities, not one optimized against

*Corresponding author.

Email address: `mateusz.miotk@ug.edu.pl` (Mateusz Miotk)

the other; the matched-utilization figures rest on five runs and are secondary. Four cross-stack rank reversals recur in both campaigns, all involving the one treatment whose MySQL driver differs, and no other pair clears the same bar, while whole-ordering agreement stays imperfect. Only that treatment reverses durably at the stated bar, and the experiment does not identify the DBMS as the cause. **Conclusion:** The protocol is a candidate: a structured, auditable proposal with one worked instantiation rather than a validated methodology, and supports no universal ranking or library-intrinsic overhead estimate. Its audit has one rater, the adapters lack independent review, and both campaigns share one host, so independent applicability remains untested.

Keywords: benchmarking methodology, database access layer, object-relational mapping, Node.js, response-time tail (p99), empirical software engineering

1. Introduction

Express.js is among the most widely used web frameworks for Node.js, and almost every non-trivial service built on it reaches a relational database through some *access layer*. Access layers trade raw performance for ergonomics, type safety, and protection from pitfalls such as the N+1 query problem [1] (Section 2). Native drivers expose the wire protocol directly (`pg`, `mysql2`), query builders assemble SQL programmatically (Knex), and object-relational mappers (ORMs) map rows to objects following patterns such as Data Mapper and Active Record [2]. Node.js back-end performance under load is a recognized empirical topic [3, 4, 5], but that literature varies the runtime and framework, not the access layer.

The most visible comparisons are vendor-authored, and the peer-reviewed ones surveyed in Section 2 each leave part of the space uncovered: breadth of products, both engines, a reported tail, or measurement through an HTTP framework (Supplement Table S51). These gaps motivate the coverage of this experiment, not a priority claim. A more basic gap is methodological: the re-

ported numbers are not established to be *comparable*, since a layer returning fewer fields or erroring faster than it works can look faster, a library’s chosen query strategy is conflated with its own cost, and a single, arbitrary load point conflates capacity, tail latency under demand, and per-request latency. Establishing that comparability is the problem this paper takes up, and it is a software-engineering one: where such benchmarks inform a technology choice, a comparison not established to be comparable can misdirect that choice rather than merely misreport a number.

This paper addresses these gaps with a *comparability protocol* (a structured, executable proposal rather than a validated methodology) and one non-vendor-authored empirical instantiation of it: an *identical* Express application served through all nine access layers over five selected access patterns, which stress distinct aspects of the access layer and are not a representative sample of Express and database workloads. Seven layers run against *both* engines under an identical schema, dataset, and pool size; two tuned native-driver baselines are disclosed reference points, not an optimization ceiling. Every (layer, engine, pattern) reports throughput and the high-load p99 under equal closed-loop demand at a fixed 50-connection point over 25 runs. That point sits near saturation, where the p99 largely tracks capacity and queueing, so we also report the tail at *matched utilization*. The study separates two estimands: the *policy-selected documented configuration* performance of each layer (RQ1–RQ3), and a *common-SQL raw-path sensitivity* contrast reporting the residual under identical SQL, which changes query formulation, protocol, preparation, round trips, and mapping together rather than isolating any one. The full harness (adapters, seed data, an autocannon-driven [6] load runner, a specification-derived expected-result oracle, differential-equivalence checks, and write-state validation) is released as an open replication package.¹

¹Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: <https://github.com/mmiotk/express-db-access-performance>. The archived release and its DOIs are given in the Data availability statement.

The experiment answers three research questions, each scoped to the benchmarked implementations under the specified operating point:

RQ1 (*performance difference*): How large is the throughput and response-time-p99 *difference* relative to the native-driver baseline, and how does it vary across the five access patterns? We keep distinct three quantities a fixed operating point conflates: *capacity*, *high-load latency*, and *latency at matched utilization*.

RQ2 (*backend-stack transfer*): Does the observed layer ordering transfer between the tested PostgreSQL and MySQL backend stacks, where changing the DBMS can also change its supported adapter and lower-level driver? The question concerns transfer, not causation: the design cannot attribute a failure to transfer to the DBMS rather than to the adapter or driver the backend brings with it. We answer it at two levels, whether a specific pair reverses durably under a stated rule and whether the whole ordering agrees, the latter reported descriptively.

RQ3 (*pattern spread*): Which of the five tested patterns shows the largest native-relative spread in this configuration?

Supplement Table S47 states, per question, the quantity the design estimates and the quantity it does *not* identify.

This paper’s primary contribution is a **comparability protocol for access-layer benchmarking**, specified normatively in Section 3.1 (inputs, stages, cell-admission criteria, interpretation, and limits). It rests on three constructs, defined there and named here. *Semantic admission* requires evidence that a treatment performed the declared task before any of its timings may be used, so a layer that returns less, writes wrong, or errors faster than it works cannot appear faster. *Treatment definition* fixes each treatment by a predeclared, reproducible selection rule, which prevents result-dependent tuning but establishes nothing about whether the configuration is optimal, common, or preferred in production. *Operating-point separation* reports capacity, high-load tail under equal demand,

and tail at matched utilization as *distinct* quantities. These constituent controls are established performance-evaluation and testing ideas [7, 8, 9], and CYNTHIA already applies differential equivalence across ORM implementations [10]; the contribution is their integration and operationalization for relational access-layer comparison. We *exercise* that discipline through an in-experiment ablation and a single-rater audit of nine external benchmarks; neither is independent validation, and Section 5 sets out what the exercise does and does not establish.

Two contributions should be kept apart. The *methodological* contribution is the protocol and its checklist: proposals for how access-layer comparisons should be conducted, reported, and audited. The *empirical* contribution is one instantiation of them, scoped to the conditions and software versions of Section 3. No result is offered as a property of a library, and the study measures no non-performance quality such as maintainability, type safety, migration tooling, or ecosystem maturity.

2. Related Work

We survey prior work along four *coverage* axes (taxonomy, engine, joint throughput and tail reporting, and vendor involvement) spanned by this experiment (Supplement Table S51), and separately audit which comparability controls each source reports. A structured scoping search, not a systematic review, of the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar (2006–July 2026) retained empirical comparisons of relational access layers or engines, with details archived in the replication package; being scoping rather than exhaustive, we scope every gap claim to the sources searched.

2.1. Object-relational mapping and the access-layer spectrum

Access layers bridge the object-relational impedance mismatch between the relational model and an application’s object graphs [11], spanning native drivers, query builders, and ORMs following Data Mapper / Active Record patterns [2] (Section 1). Torres et al. [12] survey two decades of ORM patterns; their trade-off (productivity against a hard-to-predict runtime cost) motivates what we

measure while holding the store family (relational rather than non-relational) fixed. The treatment is the configured access-layer implementation bundle, including its selected API, SQL, protocol, and mapping path; the study does not isolate a library binary from those choices.

2.2. ORM performance and data-access anti-patterns

A parallel line studies ORM performance in reverse, repairing the inefficient SQL frameworks emit. The costliest is the N+1 query problem (Section 1); it and related redundant-query anti-patterns pervade real-world web applications [13], their impact quantified in ORM-backed systems [14, 1] and often *removable* by query synthesis, batching, or caching. Closest to our design, Lorenz et al. [15] show the best mapping strategy depends on database technology, van Zyl et al. [16] that it is tuning-sensitive, and a recent study adds an energy dimension [17]. Overhead is thus configurable, not fixed, as our per-pattern results confirm, but this line never compares drivers, query builders, and ORMs head-to-head under load, our aim.

Cross-implementation ORM equivalence checking has a direct precedent. CYNTHIA generates an abstract query, translates it to equivalent queries for five ORMs, executes them over four relational DBMSs, and differentially compares the results to find correctness bugs [10]; matching results alone do not establish correctness against an independent task specification. CYNTHIA is a testing method, not a performance-benchmark admission protocol: it does not use equivalence to decide which timing cells may enter a benchmark, define a premeasurement rule for which implementation represents each library, or separate equal-demand capacity effects from matched-utilization latency. Our claim is restricted to using specification evidence and cross-implementation equivalence as benchmark admission, joined with predeclared treatment selection and operating-point separation.

2.3. Peer-reviewed access-layer comparisons and adjacent lines

Peer-reviewed head-to-head access-layer measurement is scarce, and each study leaves part of the space uncovered. Colley et al. [18] illustrate Entity

Framework-generated against hand-written SQL rather than comparing layers; two recent studies compare Prisma with optimized raw SQL, and Prisma with TypeORM and Sequelize, both only on PostgreSQL [19, 20]; and a third times internal query stages of three ORMs rather than client-observed requests [21]. Pratama and Raharja [22] do vary five access products, but framework, access path, and environment change across cells too, so no access-layer estimand is isolated. A further Express study optimizes one stack rather than comparing layers [23].

A second, disjoint line benchmarks the layers below and above the access layer without varying it. Salunke and Ouda [24] compare PostgreSQL and MySQL under standard OLTP workloads, the only dual-engine precedent found, but benchmark the *engines themselves*. On the Node.js side, work establishes Node’s web viability [3], tracks back-end drift under sustained load [4], and compares frameworks broadly [5], in each case holding the access layer fixed.

2.4. *Standard benchmarks and benchmarking methodology*

Standard benchmarks define how systems are measured: YCSB [25] for cloud-serving stores and TPC-C/TPC-E for OLTP. These target the *engine*, not the driver, query builder, or ORM this study varies, but set the bar for controlled, repeatable load. A methodological literature explains why single-metric reporting fails: Fruth et al. [8] and Raasveldt et al. [7] catalog benchmarking pitfalls, Dean and Barroso [26] show the tail drives user-perceived latency under fan-out, and Li et al. [27] trace its sources. Such measurement is often ad hoc [28] and hard to reproduce, motivating a harness built for relevance, repeatability, and fairness. Following that guidance, we report throughput *and* p99 for the same run: a pairing the general literature recommends, which we did not find in the access-layer benchmarks surveyed here.

2.5. *Practitioner and vendor benchmarks*

A larger body comes from practitioners and vendors, each authored by a party with a stake in a compared product: Drizzle’s Northwind micro-suite and

k6 load test [29], Prisma’s median-latency comparison reporting neither throughput nor any percentile [30], and IMDBench over three CRUD operations [31]. Together these span more products than the peer-reviewed literature, but none combines dual-backend-stack coverage, response-time p99, capacity characterization, and frozen source-located treatment provenance.

2.6. Positioning

Audit corpus. The nine audited sources are those retained by the same scoping search, under stated criteria rather than convenience: empirical performance comparisons of relational access layers, or direct PostgreSQL-versus-MySQL comparisons, bearing on the four positioning properties. Peer-reviewed studies, vendor suites, and practitioner benchmarks were all eligible, since the audit concerns what a benchmark reports, not who published it. Coding was performed once, by the author, from a stable locator for every judgement; **not_reported** records what the inspected source does not state, never a finding that the authors omitted the practice. The corpus is not exhaustive, so every gap statement below is scoped to it: we do not claim that no other benchmark implements these controls. Eligibility and exclusion criteria, the codebook, the screening record, and per-judgement evidence are in the online supplement and the replication package.

In the sources searched, no access-layer benchmark reports all seven protocol dimensions, unsurprisingly, since the seven are this paper’s own construction: the audit measures distance from a new standard, not failure to meet a known one. The source-located audit records partial controls rather than treating non-reporting as proof of absence (Supplement Table S37), and we found no study combining broad product coverage, both engines, joint throughput and tail reporting, and non-vendor authorship, each covering at most two or three (Supplement Table S51): the strongest claim the scoping search supports, not one of priority.

But coverage is not the contribution: the protocol operationalizes established experimental principles for this genre, not a new one. Kounev et al.’s systems-

benchmarking criteria [32], Hasselbring’s software-engineering standard [33], and Raasveldt and Mühleisen’s fair-benchmarking guidance [7] supply its principles at a deliberately general level. They leave an access-layer study to operationalize *what code represents a library*, how competing implementations are checked for semantic equivalence, and how strategy-sensitive results are contrasted. Our protocol supplies one concrete operationalization: an arbitrary but predeclared treatment policy; a common-SQL raw-path sensitivity contrast; and a per-cell gate requiring specification-conformant, mutually equivalent responses and verified write state *before* a timing is admitted. Neither element is new in itself, correctness-before-timing having precedents in SPEC validation, TPC ACID, and SQLancer [34], and equivalence checking in CYNTHIA above; the contribution is their composition into a discipline for establishing and reporting comparability among competing implementations of one task, stated in Section 1. Its evidence is the in-experiment ablation and a single-rater retrospective audit of nine external benchmarks (Supplement Table S37); this demonstrates utility and applicability, not independent inter-rater validation or invention of the underlying principles.

3. Study Design

In Goal–Question–Metric terms, the study analyzes configured relational-access-layer implementations to compare throughput and response-time p99 under PostgreSQL and MySQL back ends [35], from the viewpoint of the benchmark analyst evaluating policy-defined treatments rather than a model of observed developer behavior. This is a *controlled benchmarking experiment* on one synthetic service, dataset, and environment, not inference about ORMs as a population: treatments are assigned and executed automatically rather than observed in situ, factors are fixed by design, run order is randomized in blocks, and each cell is measured repeatedly [36, 37]. That design licenses the paired within-campaign analysis and bounds external validity to the single instantiation measured here. The experiment instantiates the protocol with

three decisions: semantic admission before a cell’s timing is used, a predeclared treatment-selection policy, and explicit operating points. Admission gates the *use* of a timing, not always its execution: several admission checks were added after the campaigns they screen (Supplement Table S48).

The primary RQ1–RQ3 treatment is the *policy-selected documented configuration*: the path selected mechanically from frozen official documentation under the rule below. The protocol does not privilege documentation; another study may preregister an expert-tuned, usage-frequency, or vendor-recommended policy. A *common-SQL raw-path sensitivity contrast* then has every portable layer execute the same SQL through its raw facility. SQL, API level, protocol, preparation, round trips, and mapping still change together, so the contrast measures sensitivity to standardizing the whole bundle; it decomposes nothing and bounds nothing (Supplement Table S42).

A fixed concurrency imposes equal *demand* on layers of unequal throughput and so drives them to unequal *utilization*. RQ1 is therefore characterized under three operating conditions, the protocol’s *operating-point separation*. *Equal external demand* is the fixed 50-connection *high-load* point covering the full matrix; a per-pattern concurrency ladder places every pattern’s throughput knee at or below 50 connections, so this point sits at high utilization of each pattern’s own capacity (median 98–100%), a capacity-and-queueing regime rather than a sub-saturation latency comparison (Supplement Table S35). The two utilization-dependent conditions need a per-layer capacity target and are shown on the deep fetch: *equal utilization* offers each layer a fixed fraction (50/70/85/95%) of its *own* estimated capacity under the coordinated-omission-corrected open-loop sweep, and *equal compute budget* is the exploratory equal-CPU check. These answer different questions and need not agree, so all three are reported. The capacity proxy $\widehat{C}_{50}$ is the median throughput of the 25-run, 50-connection primary cell and remains an estimate (Supplement Table S45); the 50% and most 70% cells stay below saturation, while 85% and 95% are read only as near-saturation trends (Supplement Tables S21–S22).

3.1. The comparability protocol

We state the protocol independently of this experiment, then instantiate it.

Comparability, operationally. Configured treatments are *comparable for a stated estimand* when each satisfies the predeclared semantic-admission and treatment-definition criteria that estimand requires, and the differences that remain between them are the differences the estimand is about. Comparability is therefore a property of a claim, not of a pair of systems, as this study’s selected-configuration and common-SQL estimands illustrate. The definition deliberately does *not* require identical SQL, round-trip counts, internal algorithms, or framework responsibilities; requiring any of these would suppress the design choices an access-layer comparison exists to measure. What admission establishes is narrower: every timed treatment performed the declared task, returned the declared result, and left the declared state, so a difference in timing is not a difference in work done. Admission is a floor, not a certificate of equivalence: admitted treatments stay unequal in ways the protocol records rather than removes, and no result below should be read as comparing otherwise-identical systems.

Two stages are mandatory, *M1* semantic admission and *M2* treatment definition, and five are recommended, *R3* to *R7*; Figure 1 states the inputs, stages, cell-admission rule, and output interpretation. They are not interchangeable: each addresses a distinct threat to interpretability, and a study may satisfy some and not others, which narrows the admissible claims rather than invalidating the reported numbers. The stages cover the admission-through-interpretation dimensions addressed here; they are not asserted to be a complete taxonomy of every threat to access-layer benchmarking. Here M1–M2 cover the full matrix, R4 every pattern, R3/R5/R6 the deep fetch, and R7 the measured source. The full normative specification (stage definitions, the cell-admission rule, applicability limits of the output comparator, the six-level compliance scheme, and the level each workload cell attains) is in the online supplement, with a machine-readable encoding in `protocol-checklist.yaml`.

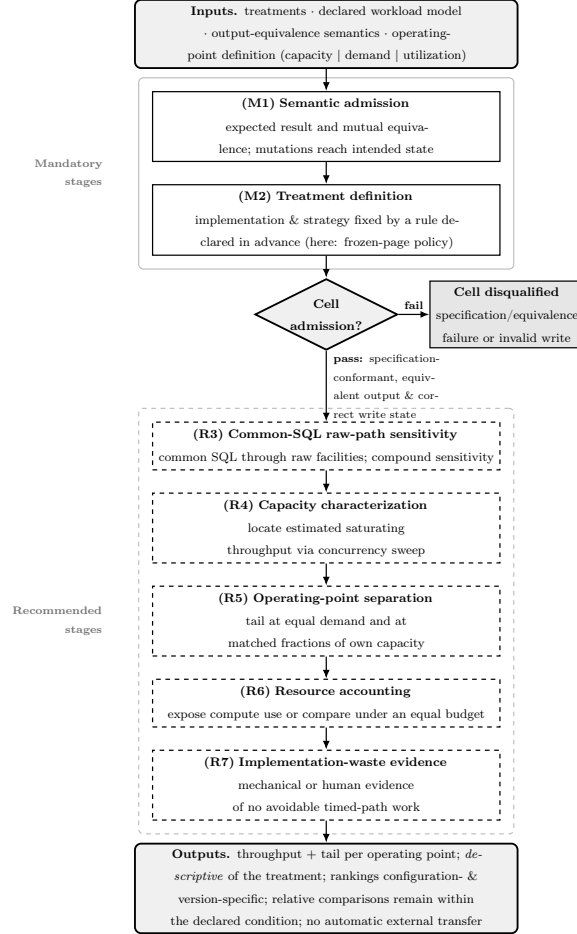


Figure 1: The comparability protocol. Semantic admission (M1) and treatment definition (M2) form the mandatory core. The recommended stages R3–R7 license common-SQL, capacity, utilization, resource, and implementation-provenance interpretations, and may be scoped to a representative workload point rather than the full matrix. A finite gate supplies task-conformance evidence; it does not prove arbitrary-input correctness. For reporting compliance, M1 and M2 together form one mandatory validity-core level and R3–R7 define five claim-specific extensions, giving the six-level scheme of Supplement Table S40.

Protocol development versus protocol validation. The protocol was not specified in full before the measurements it governs. We separate *development evidence* (the protocol finding defects in this study’s own harness) from *internal demonstration* (the worked instantiation reported here) and *external validation*, which has not been performed. Four elements postdate the accepted campaigns and are therefore retrospective, applied to archived state rather than gating the timed runs (Supplement Table S48). That the protocol exposed real defects shows the checks do work, but the defects were found by the author who designed them, so this is not independent validation.

3.2. Factors and treatments

We vary three factors. The *access layer* has eleven levels across three tiers. Two native drivers (`pg`, `mysql2`) and their two engine-specific tuned baselines each run only on their own engine; the query builder Knex and the six ORMs (Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM) run on both. The tuned baselines are disclosed reference points, not selected treatments, leaving *nine* policy-selected layers and $4 + 7 \times 2 = 18$ layer $\times$ engine cells across 5 patterns, so 90 measured configurations. We classify a library by its dominant interface, not its self-description (Section 1). The nine are a deliberately broad product cross-section, not balanced sampling of the three tiers and not an exhaustive catalogue; inclusion and exclusion criteria are in `METHODOLOGY.md`. *Non-vendor authorship* means only that no evaluated library’s maintainers were involved; the consequential author choices (selection, schema, pool size, tuning) remain and are disclosed. All adapter code and treatment assignments were produced by the single author, a limitation analysed in Section 6.

The *backend-stack* factor has two nominal DBMS levels (PostgreSQL 18.4 and MySQL 9.7.1), but switching it can also switch the supported adapter, driver, wire protocol, and defaults, so RQ2 estimates transfer between stacks, not a pure DBMS effect. The *access pattern* has five levels, realized as HTTP endpoints (Supplement Table S39). Supplement Table S29 records which factors are held constant, vary, or are uncontrolled on this single virtualized host, the

reason we report this configuration rather than general library performance.

3.3. Objects: schema, workload, and data

The workload is a minimal blog domain (`authors 1-* posts 1-* comments *-1 authors`). The five endpoints implement five selected access patterns [25]: a keyset page (primary-key cursor, not a large `OFFSET`), a single-row read, a one-to-many list, the deep fetch (a post with its author and every comment, each with its own comment-author), and a per-author aggregation (Supplement Table S39). A deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) is loaded by the native driver, independently of any layer under test, so both engines receive byte-identical row values. The working set fits in memory, so measurements emphasize the access layer’s per-request instruction, protocol, and mapping cost, while engine-side execution, buffering, locking, and logging remain in every measurement [38]. Each request’s target identifier is drawn uniformly from the seeded range, spreading load across the working set; a skewed, production-like distribution is a planned variant (Section 6).

3.4. The adapter contract and $N+1$ control

Every access layer implements the same factory interface, so the server and load runner are layer-agnostic and each returns an *identical result shape*. Each avoids the deep-fetch $N+1$ problem by its selected mechanism, a hand-written join for the native drivers, Knex, and Drizzle, and relation eager loading for the data-mapper ORMs. The comparison is thus of each layer’s *selected configuration* under a fixed adapter-selection rule, not a fixed query plan: a measured difference reflects the SQL generated, the round-trip count, and result materialization, all captured by server-side logging (Supplement Table S2).

The API was fixed by a rule recorded before the accepted campaigns (2026-07-22; Supplement Table S48) and applied unchanged: the eager-loading API each layer’s version-compatible official documentation presents first, ties broken toward the lower-level API the library documents as its base layer. This is an intentionally artificial, predeclared *policy*, not evidence that the chosen API is

performance-optimal, the most common production choice, or what an expert would deploy; it is one instance of the protocol’s predeclared treatment rule, which privileges none (corroboration and limits: Table S33, Section 6). Reproducible selection is also not yet independently *reproduced*: all four result-blind review slots stand pending, so whether a second analyst applying this rule to the same frozen pages would select the same APIs is untested.

Supplement Table S16 and the rubric (`notes/documentation-selection.md`) record, per layer, the selected API with its documentation page, version, and URL, its round-trip count, the documented alternative not used, that logging, hooks, and validation are off, and the freeze date. The artifact ships the exact pre-freeze page captures with timestamps and hashes, so reconstruction does not depend on today’s website; a capture proves the page state at its recorded timestamp, not that it was unchanged through the freeze. The conflict and tie-break rules are in the online supplement.

For the protocol’s *common-SQL raw-path sensitivity contrast*, every adapter also executes the same two-statement deep-fetch SQL and row mapping through its raw-SQL facility, with the same **EXPLAIN** plan per engine; server-side capture confirms the statements are byte-identical across every layer on both engines, while the raw facility, preparation, and wire protocol still differ (Supplement Table S42). A secondary loading-strategy sensitivity switches only between documented, byte-identical deep-fetch alternatives; it does not redefine the primary policy-selected treatment or estimate a library-intrinsic effect (Supplement Table S18).

Read admission has an expected-result layer independent of any timed reference: `bench/verify-spec.mjs` replays the documented seed PRNG and fan-out fixture in memory, without querying a database or importing an access layer, and derives exact fields, values, ordering, graph membership, and aggregates. All nine engine-compatible adapters pass 36,540 expected-result checks per engine, 73,080 in total. A separate differential layer establishes that implementations agree, and a companion check validates database *state* rather than HTTP status, confirming for every adapter on both engines that the primary insert wrote ex-

actly the requested values and identifier and that the exploratory transactional method commits atomically or rolls back to no partial state; all declared methods pass. Both layers provide finite evidence for the declared task and tested inputs; neither is presented as proof for arbitrary inputs. The canonical constructors, per-adapter scope, fan-out and timestamp conventions, and fixture details are in the online supplement and `experiments/METHODOLOGY.md`. The read cross-check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; Section 5).

3.5. Controls

Four controls hold the shared conditions fixed (connection pool, logical task, physical dataset state, and observer) so the *configured access-layer implementation bundle* is the only *deliberately varied* factor. They do not isolate the library alone: the bundled differences in SQL, query count, protocol, loading strategy, and hydration are the treatment, not confounds to remove (Supplement Table S42). (i) *Pool size* is fixed at ten connections for every adapter, so the comparison reflects the access-layer choice, not pool tuning [39, 40]; because the generator opens 50 connections against that pool, requests queue throughout, with instrumented layers reporting mean pending depth 29–38, unobserved for Prisma and MikroORM, which expose no pool metric. (ii) *Common task semantics*: each endpoint implements the same declared input–output task, while SQL text, query count, plan, protocol, loading strategy, and hydration remain measured properties of the treatment. Aggregation is the exception, with a common correlated-subquery formulation. (iii) *Write isolation*: the declared reset removes generated rows before warm-up, measurement, and the next cell. A post-review state oracle and a cross-engine gate exposed two defects in the historical harness, an allocator that could let generated rows persist and a shared-fixture PRNG that gave different comment-author assignments across engines; the corrected harness pins the allocator, verifies exact base and fan-out counts by idempotent preflight, and materializes the fixture once so the joined representation is byte-identical across engines. The affected pilot was rejected

and a corrected-state campaign remeasured all five patterns for all 18 compatible treatment-stack pairs under one version of the instrument; the historical MySQL range-scan, aggregation, and insert cells are not used as clean evidence. Reset mechanics, sentinel values, and fixture hashes are in the online supplement and `experiments/METHODOLOGY.md`. (iv) *Observer separation*: the HTTP load generator runs in a process separate from the server.

3.6. Measurement procedure

Load was generated with `autocannon` [6], a closed-loop (constant-concurrency) HTTP/1.1 generator [41] recording a latency histogram. The accepted corrected-state primary matrix contains 90 configurations with 25 repeated runs each; the superseded campaign remains provenance only. Each replicate randomizes adapter-engine blocks and boots one fresh server process per block, so no application process crosses an adapter, engine, or replicate boundary; read endpoints within a block share that process, and database buffers plus host state are not reset. Each endpoint receives a discarded 15-second warm-up followed by one 12-second measurement at 50 connections, and a forward/reverse-order write-cell check found no detectable order effect. The campaign records its ID, deterministic order seed, environment fingerprint, and exact source manifest; process, teardown, and rebuild mechanics are in `experiments/METHODOLOGY.md`.

Within a replicate, every layer and engine draws target identifiers from one generator seeded on endpoint and replicate, so all face the same identifier distribution, though not an identical served sequence; the paired analysis is licensed by the block design, not by sequence identity. The inferential unit is the endpoint-level run, individual requests being subsamples, and we report the median of the 25 replicate values of throughput and of the run-level percentiles p50, p90, p97.5, and p99.

Throughout, *p99* denotes this *median run-level p99*, not the p99 of the pooled request distribution, which the design does not estimate. Ten 60 s re-measurements of the deep fetch give 12-versus-60-second rank correlations of

1.00 on both engines (Supplement Table S23): endpoint- and campaign-specific sensitivity evidence, not a general claim that 12 s suffices for p99 estimation. We sampled server CPU and peak RSS over the process tree, with database and load-generator CPU separately. Inserts were measured under each engine’s *default* durability, the regime a deployment runs unless relaxed; a labelled secondary run relaxed all of these as a compound durability sensitivity (Supplement Table S3). To characterize behavior under load we swept the deep fetch on a separate 1–256-connection ladder for every layer and engine [42], and a constant-arrival generator drove it under a fixed offered rate rather than fixed concurrency, measuring from intended send time, a coordinated-omission-corrected design [43, 44] checking the closed-loop tail’s sensitivity to a different load model (Section 4).

3.7. Analysis

We separate *primary* outcomes, which carry the research-question claims, from *secondary* and *exploratory* outcomes reported descriptively as controls and sensitivity analyses (Supplement Table S34); the primary outcomes are throughput and the closed-loop p99 on the five patterns against both engines at the fixed operating point. Because absolute throughput is hardware-specific, we analyze *relative* differences within an engine, a pattern’s *spread* being the untuned native driver’s median throughput divided by that of the slowest layer on that pattern. We report medians over the 25 runs, the coefficient of variation as a stability signal [9, 45], and a fixed-seed percentile bootstrap 95% CI for the median; these intervals capture within-campaign, run-to-run variability on this host and configuration, not variation across machines, versions, or deployments (Section 6). Because the design is a *randomized block*, adjacently ranked layers are compared with *paired* tests on their per-replicate throughput ratios, a two-sided sign-flip permutation test cross-checked with Wilcoxon signed-rank, reporting the geometric-mean paired ratio, its bootstrap CI, the paired dominance, and a bootstrap-interval equivalence assessment against an author-selected $\pm 5\%$ margin for the closest pair (Supplement Table S56); we foreground these effect sizes and interval estimates over *p*-values [46, 47]. Because the seven adjacent pairs

follow the *observed* ranking, their significances are a post-hoc, descriptive robustness check, not a confirmatory family, and the Wilcoxon cross-check reuses the same data and design rather than confirming it independently. All tests operate on the 25 paired run-level values; no interval is computed over pooled requests. Randomization is over *measurement order*, so it supports temporal balance, not causal identification: the treatments are pre-existing implementations, not interventions assigned to units, and no result should be read as a causal effect of a library on throughput. Rank correlations are likewise descriptive, the seven layers being the benchmarked population of configurations rather than a random sample. Adjacent-layer p99 gaps of one or two milliseconds sit at the millisecond measurement floor and are not read as real; conclusions rest on the large-ratio patterns, chiefly the deep fetch. The exchangeability assumption and its argument, the full construction, and the statistical-methods notes are in the online supplement and `notes/supplement-methods`; paired p99 comparisons are Supplement Table S30.

3.8. *Experimental setup*

All measurements were taken on a single host, a virtualized Intel Xeon Gold 6140 instance (16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.18.0; Express 5), PostgreSQL 18.4 and MySQL 9.7.1 local. Fourteen labelled secondary experiments ran 2026-07-16 to 2026-07-18, the accepted corrected-state primary matrix overnight on 2026-07-24, and the same-host validation campaign on 2026-07-25, each dated by its archived environment capture (Supplement Table S48). The secondary experiments therefore predate the accepted primary matrix rather than sharing its window, and the clean-evidence scoping applies to them as it does to the superseded pilot. Each library is pinned to the *latest stable release compatible with the harness adapter contract* as of the freeze date (2026-07-15), recorded from the committed lockfile and an automated environment capture (Supplement Table S11); only Sequelize invokes the earlier-release exception. Because access-layer performance is version-sensitive, this pins a dated snapshot rather than a permanent ranking (Section 4).

3.9. AI-assisted research tooling

AI assistance covered research software and data analysis, not language editing alone, so it is documented here as part of the method; the publisher’s separate manuscript-preparation declaration appears before the references. The sole assistant was Claude (Anthropic), in three model versions; no other vendor’s assistant contributed software, analysis, or measurements, and OpenAI Codex was used only after the manuscript was otherwise complete, for two independent critical pre-submission reviews. Assistance covered the eleven adapters and harness scaffolding, the table and figure generators, the statistical estimators, and manuscript prose, and extended beyond implementation to proposing candidate analysis procedures, each a named published method, among which the author selected and verified. Author review is not the safeguard: each category carries a check independent of the generated code, namely unit-tested estimators, per-cell tracing of every measured value to an archived record, and the specification-derived oracle gating the adapters. Their limit is exact. They establish that the estimators compute what they claim and that each measured number came from an archived measurement; *no* check bears on whether the chosen procedures suit the question, or whether the claims drawn from them are sound, and those rest on author judgement alone. Per-commit provenance, the affected components, the individual validation checks, and the full extent of assistance in analysis are in the online supplement.

3.10. Replication package

The harness is released so the full matrix re-runs with one command against a containerized or local PostgreSQL and MySQL. Two reproduction statuses are kept distinct: an author-run *reconstruction* regenerates the archived tables from the archived raw records and verifies their hashes, repeatable by a reader without a database, whereas *independent reproduction* would require another team re-executing the measurements on its own hardware and none has been performed (Supplement Table S31).

4. Results

Tables report throughput (req/s, higher better) and tail latency (p99 ms, lower better) per access layer and engine; the deep fetch is in the main text (Table 1) and the other four patterns in Supplement Tables S52, S12, S13, and S15. Each native driver has an untuned policy-selected row and a tuned one marking a hand-tuned reference point, not one attained without code changes. We compare *relative* differences within an engine, and two estimands recur (Section 1): the selected configured treatment’s performance, which is not library-only overhead, and the common-SQL raw-path sensitivity contrast (Supplement Table S14).

Table 1: Deep fetch: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within the source campaign of each cell, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

Layer	Category	PostgreSQL		MySQL	
		req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	3638 [3602–3662]	18 [17–20]	–	–
mysql2	native-driver	–	–	2416 [2413–2425]	28 [27–28]
pg-tuned	native-tuned	3700 [3656–3773]	18 [16–19]	–	–
mysql2-tuned	native-tuned	–	–	2398 [2386–2419]	25 [24–26]
knex	query-builder	2431 [2415–2459]	27 [25–28]	1982 [1963–2006]	30 [29–31]
drizzle	orm	1829 [1815–1846]	34 [33–35]	1301 [1290–1308]	48 [46–49]
prisma	orm	1175 [1160–1179]	52 [51–53]	630 [623–638]	91 [89–94]
sequelize	orm	978 [973–983]	60 [60–61]	880 [871–886]	68 [67–68]
typeorm	orm	1219 [1205–1233]	49 [48–50]	1017 [1008–1030]	57 [56–58]
objection	orm	1017 [1011–1029]	56 [54–58]	836 [824–849]	70 [68–70]
mikroorm	orm	519 [512–528]	108 [107–112]	472 [464–475]	121 [118–126]

4.1. RQ1: performance difference relative to the native baseline

The native implementation leads nine of the ten engine–pattern comparisons. The exception is the MySQL insert, where the four fastest treatments span

1,766–1,816 req/s with broadly overlapping intervals and the native driver ranks fourth (Supplement Table S52); that ordering is practically unresolved and we read no leader from it. Current ORMs sit mid-to-low on the read patterns, Prisma near the bottom of point reads and aggregation on both engines (Table 1; Supplement Tables S12 and S15), retiring an earlier headline of Prisma tying the native driver at a large CPU premium (Section 5).

The tuned baselines track their untuned counterparts within about 2% on the deep fetch. The one material gain is PostgreSQL aggregation, where named prepared statements add 11% (7,583 vs. 6,807); on MySQL the binary-protocol baseline is *slower* than `mysql2` on every pattern, every difference lying inside the $\pm 5\%$ margin. Both are therefore disclosed reference points, not optimization ceilings.

On the **deep fetch** (a post, its author, and all comments with their authors) the native-relative spread is the widest measured, $7.01\times$ on PostgreSQL (pg 3,638, 95% CI [3,602–3,662], down to MikroORM’s 519 [512–528]) and $5.12\times$ on MySQL (`mysql2` 2,416 [2,413–2,425] down to MikroORM’s 472 [464–475]). The native driver leads on both engines, the data-mapper ORMs trailing furthest (Table 1); the comparison is paired (Section 3), every adjacent MySQL step large relative to its bootstrap interval (Supplement Table S36). A sensitivity equivalence assessment does not establish $\pm 5\%$ equivalence for the closest adjacent selected pair on either engine (Supplement Table S56). Aggregation is the least layer-sensitive pattern: every layer forwards a single query and avoids the deep fetch’s nested-graph materialization, though the design isolates neither feature as the cause. The remaining per-pattern spreads are collected under RQ3 below.

4.2. Common-SQL raw-path sensitivity contrast

With statement text and database plans held common across raw paths, though API level, wire protocol, preparation, round trips, and mapping still differ, the deep-fetch native-relative spread is far smaller in a separate ten-run campaign, $1.71\times$ (PostgreSQL) and $2.03\times$ (MySQL) against the selected-

configuration $7.01\times$ and $5.12\times$ (Supplement Tables S42 and S14). The selected paths therefore differ more than these common-SQL raw paths, but the compound contrast assigns no share to any mechanism. A loading-strategy check argues against a mere selected-strategy artifact, forcing the join-selected ORMs to select-in making them slower (Supplement Table S18). Neither comparison isolates a library-only cost.

4.3. RQ2: backend-stack transfer of the ranking

Switching the backend changes a bundle that can include the DBMS, supported adapter, lower-level driver, wire protocol, and defaults. RQ2 therefore concerns *backend-stack transfer*, not a causal DBMS effect. In the primary campaign, the seven portable layers have stack-ratio heterogeneity on every pattern (blocked interaction $F = 127.1\text{--}458.5$, permutation $p = 1/20,001$, the resolution floor at $B = 20,000$; Supplement Table S28), but this within-campaign test does not establish durable rank reversals.

A second same-host campaign re-runs 25 repetitions of the seven portable layers for deep fetch, aggregation, and insert under an independently drawn block order (Supplement Table S46). It re-randomizes execution order only: the identifier generator is seeded on endpoint and replicate, so it replays the same draws and cannot detect sensitivity to the workload sample. Deep-fetch ranks reproduce exactly on both stacks ($\rho = 1.00$); aggregation reproduces 5/7 PostgreSQL and 7/7 MySQL positions. MySQL insert is less stable: 3/7 positions, $\rho = 0.82$, and the leader changes from Drizzle to Knex.

We therefore promote a cross-stack reversal only when it occurs in both campaigns, exceeds the 5% practical margin on both stacks, and paired bootstrap intervals exclude equality in opposite directions in both. The margin was fixed before these campaigns (Section 3), but the composite rule was specified after both completed, a post-hoc screen applied uniformly to all 21 layer pairs on each of three patterns. It applies no familywise correction: recurrence screens one-campaign noise, but the campaigns share a host and the same seeded draws, so it does not bound familywise error across the 63 comparisons. Applying Holm

afterwards to an intersection-union test retains all four (Supplement Table S55), correcting the per-stack evidence but not the direction and margin filters; they are a stable subset of an exploratory comparison, not a prespecified test. Four pairs meet all criteria, sized with paired intervals in Supplement Table S54: Prisma exceeds Objection and Sequelize on PostgreSQL deep fetch but trails both on MySQL, exceeds Objection on aggregation but trails it on MySQL, and exceeds MikroORM on insert but trails it on MySQL. All four therefore involve one treatment, and it is the one whose lower-level driver differs between the stacks: Prisma runs the MariaDB adapter on MySQL because it ships no `mysql2` adapter, while every other layer uses `mysql2` (Section 6). The recurrence is consistent with backend-stack sensitivity as RQ2 defines it, but it cannot be separated from that driver substitution, and it is not evidence about ORMs as a group. Other changes in close ranks, including the identity of the MySQL insert leader, remain exploratory.

The rule is applied per pair, so the 15 non-Prisma pairs were screened in the same pass and none was promoted, each failing the 5% margin or the recurrence requirement. Holding Prisma out promotes no other pair and otherwise changes only the whole-ordering correlations, which move in both directions and stay imperfect on both stacks either way (Supplement Table S49). The promoted-reversal evidence therefore rests entirely on the treatment carrying the driver substitution. What the contrast licenses, and what it cannot identify, is set out in Section 5.

The magnitude of stack transfer is also treatment-specific. Primary-campaign PostgreSQL÷MySQL ratios range from 1.10 to 1.84× on deep fetch, 1.27 to 1.82× on aggregation, and 1.55 to 3.06× on insert (Supplement Table S28). These ratios and the recurring reversals show that the observed ordering cannot be assumed to transfer unchanged between the two tested stacks. They do not allocate the difference to the DBMS, adapter, driver, protocol, or defaults.

The primary MySQL insert medians illustrate why fine ordering is not a headline: Drizzle, Objection, Knex, and the native driver lie between 1,766

and 1,816 req/s, while several replicate distributions are multimodal or heavy-tailed and reach 13.6% within-campaign CV (Supplement Figure S1). Prisma is the slowest policy-selected treatment at 894 req/s, giving a $1.98\times$ native-relative spread on a broad interval [1.65–2.09]. Under relaxed durability the native MySQL and Knex paths gain $2.42\times$ and $2.07\times$, ranging $1.02\text{--}2.44\times$ across treatments; concurrent wait-event sums are consistent with a shared commit-path contribution but are not a causal time fraction (Supplement Tables S3 and S19). PostgreSQL insert spans $3.18\times$ (6,018 to 1,894 req/s), changes at most 14% under relaxed durability, with p99 12–32 ms against MySQL’s 105–129 ms. These compound interventions do not decompose backend-stack cost.

4.4. *RQ3: which pattern spreads widest*

The primary native-relative spreads on PostgreSQL are deep fetch ($7.01\times$), keyset range scan ($4.22\times$), insert ($3.18\times$), point read ($2.83\times$), and aggregation ($1.82\times$). On MySQL they are deep fetch ($5.12\times$), range scan ($3.88\times$), point read ($2.14\times$), insert ($1.98\times$; max/min $2.03\times$), and aggregation ($1.87\times$). The tested deep fetch has the largest spread and aggregation the smallest on both stacks. A full concurrency sweep places the median knee at 8–32 connections and the median utilization at 50 connections at 98–100% for every pattern-stack combination; the lowest individual values are 87–88% for two PostgreSQL patterns (Supplement Table S35). The fixed point is therefore a high-utilization comparison throughout, though not an identical fraction for every layer. An exploratory fan-out sweep (0–500 comments) leaves multipliers flat on MySQL but widens the PostgreSQL spread from $6.0\times$ to $11.7\times$ (Supplement Table S53).

4.5. *Tail latency*

At equal closed-loop demand, deep-fetch p99 spans 18–108 ms on PostgreSQL and 28–121 ms among the policy-selected layers on MySQL (Table 1). The run is the analysis unit, so we use the median of 25 run-level p99 values and show their raw spread. Ten 60-second repetitions preserve the ordering (Supplement Table S23), which is endpoint-specific evidence, not a general sufficiency claim.

At the stable nominal 50% capacity target, coordinated-omission-corrected p99 is 2.3–4.9 ms on PostgreSQL and 1.9–5.5 ms on MySQL, a different operating point and estimand, not an optimization applied to the systems; most 70% cells remain similarly small, while the 85% and 95% results are unstable near-saturation observations supporting no convergence claim (Supplement Tables S21–S22, S43 and S45). The stable 50% result shows the large equal-demand p99 spread contains a material capacity-and-queueing component; it does not identify its share or imply a deployment-general sub-saturation bound.

The two operating points also *order the layers differently*, the sharpest consequence of separating them, though they differ in load generator and model too (closed-loop, 25 runs; open-loop, 5), so the reordering is not the operating point alone. Between the equal-demand and 50%-utilization p99 orderings, Spearman ρ is 0.64 on PostgreSQL and 0.29 on MySQL (Supplement Table S43). The native driver moves furthest on MySQL: fastest at equal demand on both engines, it falls to third of eight on PostgreSQL and seventh of eight on MySQL once each layer is held at half its own capacity, a reversal large relative to the five-run spread (**knex** 1.9 ms [1.8–2.2] against **mysql2** 4.7 ms [4.0–5.6]). On PostgreSQL the native and leading portable ranges overlap, so only the MySQL inversion is claimed. Reporting a tail at one arbitrary load point would have recorded the native driver as the tail leader on both engines: secondary evidence that operating-point separation changes a conclusion rather than only its caveats.

4.6. Measurement stability and effect sizes

Deep-fetch throughput CV reaches 4.55%, much smaller than its multi-fold spreads, and the largest primary throughput CV is 13.6% on MySQL insert, whose raw replicates are shown because a median and interval can hide regime mixtures. All 90 cells contain exactly 25 run-level samples and zero timed errors, timeouts, or non-2xx responses, enforced by a fail-closed roster validation. Predeclared native-driver contrasts are foregrounded and ranking-selected adjacent comparisons remain descriptive; paired bootstrap intervals characterize only within-campaign variation. Differences of about 1 ms in p99 and through-

put ratios inside the predeclared $\pm 5\%$ practical margin are not promoted as substantive ordering.

5. Discussion

Version-sensitivity. An earlier campaign found Prisma (then 5.22, on an in-process Rust query engine) tying the native driver on the deep fetch while drawing about five application cores. The *identical* harness on the re-frozen toolchain did not reproduce it: application CPU now stays near one core and Prisma sits at mid-to-low throughput (Section 4). The non-reproduction is a fact, its cause a hypothesis, since the re-freeze moved the whole toolchain at once, so the headline is version-sensitive [16].

Implications for practice. RQ1 and RQ3 agree the selected-configuration difference is specific to the access *pattern*: the native-relative spread is widest on the deep fetch, where a layer must hydrate rows and resolve associations rather than merely issue SQL (the object-relational impedance mismatch [11] made concrete), and narrowest on aggregation. Round-trip count alone does not explain throughput: single-query MikroORM is slowest on the PostgreSQL deep fetch while two-query Drizzle outruns four-query Prisma (Supplement Table S2; Table 1). Result-graph materialization and the other bundled implementation differences are plausible contributors, but the study does not isolate them.

Single-row inserts and backend-stack transfer. Single-row insert behavior is both stack- and treatment-specific, and the durability and wait-event controls reported there are consistent with a shared commit-path contribution, not with attributing a performance floor or a fixed share to the DBMS (Section 4).

The second same-host campaign preserves PostgreSQL insert ranks but few MySQL positions (Section 4), so we claim no durable fine ordering for MySQL insert. The recurring, material insert reversal is narrower: Prisma exceeds MikroORM on PostgreSQL and trails it on MySQL in both campaigns,

with paired intervals excluding equality in opposite directions (Supplement Table S54).

Every promoted reversal involves Prisma, whose MySQL path uses the MariaDB adapter rather than `mysql2`, so backend-stack sensitivity here is inseparable from that driver substitution and describes one treatment, not ORMs as a class. *Observed:* every reversal clearing the bar involves the one layer whose MySQL driver differs. *Interpretation:* at the stated bar only the asymmetric treatment reverses durably, while whole-ordering agreement stays imperfect either way, reported descriptively rather than as a further reversal claim. *Not identified:* the design cannot attribute the reversals to the DBMS rather than the adapter, the driver, or Prisma’s implementation, because holding out the treatment holds out the substitution; the hold-out followed observing that all four involved one treatment, and its criterion is a design fact fixed independently of the outcomes. This supports backend-stack sensitivity in the tested implementations, leaving cross-host transfer unresolved. A five-replicate, one-layer-per-tier six-statement transaction remains exploratory and does not extend the single-row-insert finding to writes (Supplement Table S8).

Methodological lessons: a checklist for access-layer benchmarking. Several failures changed or could have changed the comparison; we distil them into a reusable checklist (`notes/supplement-methods`): an **aggregation fan-out** and a driver result-shape mismatch, both caught by specification and differential admission; **OFFSET pagination** collapsing on MySQL, easily misread as an access-layer weakness; **write-state contamination**, where MySQL allocated inserts below the cleanup floor; and a **query-formulation confound** where a mis-scoped pre-aggregation re-scanned the whole table, a query-plan artifact masquerading as an access-layer effect. The write path needs its own gate: MikroORM 7’s dropped `persistAndFlush` made every insert throw a 500 faster than a real one, briefly *leading* MySQL until the non-2xx counter flagged it. Because a 2xx alone would miss a *silently* incorrect write, the gate also validates post-write state. None is schema-specific; each extends the

database-benchmarking pitfalls of Fruth et al. [8] and Raasveldt et al. [7] to the access-layer sub-genre.

Evidence for the protocol contribution. Each stage had a concrete consequence: an independent seed specification caught shared state and fixture defects that mutual equality would miss, treatment provenance made the loading-policy sensitivity auditable, capacity characterization changed the tail interpretation, and the common-SQL path narrowed but did not decompose the selected-treatment spread (Supplement Table S50). Applied to the nine external sources, the codebook yields claim licenses rather than a binary quality score (Supplement Table S37).

This demonstrates that the protocol is *executable*: it admits and rejects concrete cells, changed the reported interpretation at every stage exercised here, and codes an external corpus to a reproducible per-stage result. It does not demonstrate *reusability by another person*, and two limits sharpen that: no source was coded *satisfied* on any stage, so the codebook’s upper range is untested, and agreement against this rater would measure agreement with the protocol’s own designer. The codebook, audit record, agreement scorer, and result-blind forms are released so the question can be settled against this study. The claim is therefore a *candidate* discipline and reusable artifact, demonstrated once: not invention of the constituent controls, not a complete theory of benchmarking validity, and not a validated methodology.

Practical guidance. The synthesis follows from RQ1–RQ3 and describes the benchmarked implementations in the tested configuration (Section 6), not access-layer categories in general. Because the fan-out sweep varies breadth (0–500 children) but not depth, schema, or query mix, the deep-fetch peak locates a difference among five probes on one synthetic schema: an observation to re-measure on the target workload, not a law. Where a hot path resembles the tested deep fetch the access layer is consequential and worth benchmarking, the thin paths (native driver, Knex) leading here for throughput and for p99 at high load, though at matched utilization the MySQL native driver falls to

seventh of eight (Section 4). Where the tested workload is read-simple, or for the tested MySQL single-row insert, the observed layer spread is smaller. These are performance findings only: the choice also turns on qualities this study does not measure (Section 1), which may outweigh them, and the common-SQL results are a compound sensitivity contrast on the selected-configuration cost, not a licence to use raw SQL.

6. Threats to Validity

We organize the threats along the four standard validity categories [48, 36]. The supporting measurements and mechanics for each are in the online supplement; every limitation is stated here.

Construct validity of the measures. Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) at the Express endpoint, not query-execution time at the driver boundary. This is deliberate, capturing the full request round trip, though overhead attributed to the *access layer* may partly reflect Express routing and serialization. We hold those near-constant: every adapter satisfies a documented *adapter contract* and funnels rows through shared canonical constructors whose standalone cost is too small to explain the multi-fold gap, and a fixed-payload endpoint puts the shared framework-and-serialization floor at least three times above the fastest deep-fetch cell, so no cell is framework-bound. The specification-derived oracle and the differential checks establish finite conformance and mutual equivalence (Section 3); the differences arise in how each layer reaches an identical response, not in what is returned. The reported single-row-insert finding uses each engine’s *default* vendor-standard durability, not one we relaxed, so those insert spreads (Supplement Table S52) describe the tested default deployment rather than a tuning choice; the relaxed-durability sensitivity is a compound intervention and does not decompose its cause (Section 5, Supplement Table S3).

Construct validity of the treatment. RQ1’s treatment is each layer’s *policy-selected documented* implementation and eager-loading strategy: an operational

rule not validated against surveyed, typical, or performance-tuned usage. It is an intentionally artificial, predeclared selection policy, not a behavioral practitioner model. The frozen pages, conflict rule, and assignments make the decision auditable, but the single author both defined and applied it, and two blind selection reviews are prepared and still pending. Where the policy differs from a performance-tuned configuration, RQ1’s spread mixes the library’s execution machinery with that strategy choice, made material by the one-to-four statement range on the deep fetch (Supplement Table S2). Three sensitivity checks qualify it, and none of them decomposes the spread: the preserved source-page provenance is provenance, not evidence of frequency or idiomatcity; the *common-SQL raw-path sensitivity contrast* is a compound contrast, not a bound on how much comes from the loading strategy versus the library machinery; and the loading-strategy check shows the selected path is the *faster* strategy for the join-selected ORMs, with Objection’s slower selected select-in path on MySQL disclosed rather than corrected. RQ1 therefore answers what this policy yields, not a library-only overhead.

Internal validity. Each adapter follows the registered policy and passes specification-derived and differential checks, but all were implemented by one author. A semantically correct yet needlessly allocating, converting, or serializing adapter would pass; no independent human implementation review is claimed. A result-blind author self-audit found two avoidable timed-path operations before campaign acceptance, both removed without changing the declared treatment or SQL, after which the admission chain passed again and measurement restarted from repetition 1 (Section 5); because the implementer performed it, this is author review.

Both are now detected mechanically instead, which is why R7 requires evidence rather than a reviewer: a static check reports values acquired and never read, and a runtime gate rejects any cell writing to `stderr` or `stdout` during a measured run. The runtime gate postdates the reported campaign and did not gate it, so absence of output there rests on author observation. Neither covers

waste that is silent *and* invisible in the source, so the residual narrows without closing: a correct, policy-conformant adapter uniformly slower than a differently written one would pass every check. We report it as open, not discharged. The same caution applies more widely: four admission elements postdate the accepted campaigns (Supplement Table S48). Because they replay a deterministic seed and inspect archived state, they can invalidate archived results, and did so for the superseded pilot; but they did not gate the timed runs as those runs executed, so a defect observable only while a run was in progress would have escaped them.

Per-cell warm-up phases exceeding the slowest layer’s cold-start stabilization absorb JIT, pool-fill, and plan-cache effects [49], and two further controls address non-layer confounds: one correlated-subquery aggregation plan and the corrected-state campaign’s verified reset before each write cell. The historical MySQL allocator defect, the corrected reset, and the seeded identifier design are described in Section 3; their consequence here is that the affected historical cells are not used as clean evidence, all five patterns having been remeasured in one campaign. A forward-versus-reversed write-cell-order check found no detectable order effect, so order does not confound layer identity with run position. A residual risk is coordinated omission, which can under-report the tail at saturation [8, 43]; the corrected constant-arrival sweeps show small tails below measured capacity and sharp growth near it (Supplement Tables S6 and S17).

External validity. All measurements come from a single schema, dataset size, and 16-vCPU virtualized host, a pool fixed at ten, and specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1; latest compatible stable versions as of the 15 July 2026 freeze, Supplement Table S11). The fixed pool does not appear to drive the ranking: the pool-size frontier preserves the ordering from 2 to 50 connections and a mixed read/write workload preserves the read ordering, both exploratory and on four layers, though at a pool of one Prisma overtakes Knex on PostgreSQL (Supplement Tables S7, S25, S26). Two choices are disclosed rather than resolved: Sequelize on its stable 6.x line, and Prisma’s MySQL path

on the MariaDB adapter, an implementation $\times$ engine asymmetry. The memory-resident working set is a hot-cache regime that makes access-layer overhead more visible; as a workload becomes I/O-bound the spreads may compress toward $1\times$.

Same-host checks do not bound an independent substrate: a post-restart rerun of six representative cells moved absolute throughput materially against the campaign prevailing when it ran, which is the superseded pilot rather than the accepted campaign. The corrected-state 25-run campaign repeats all five patterns with a new randomized order and environment fingerprint on the same host. Cross-campaign agreement or disagreement is reported directly; it reduces dependence on one overnight ordering but does not address host, hypervisor-neighbor, CPU-steal, or frequency transfer. No empirical ordering is intended to travel beyond the tested campaigns, and close cross-backend rank reversals are exploratory unless their direction repeats. Every promoted RQ2 reversal involves the single treatment whose MySQL driver differs, and a leave-one-out repetition of the promotion procedure without it promotes none (Supplement Table S49); the promoted-reversal evidence is therefore conditional on that driver substitution, which the design cannot separate from the treatment or from the DBMS. The protocol evidence is also bounded: its external audit has one rater, so general independent applicability remains to be tested.

Conclusion validity. Our analysis (Section 3) reports medians with the coefficient of variation and within-campaign bootstrap 95% intervals (repeatability, not population-general) and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios. Two bounds reinforce the ordering: the widest spreads dwarf the deep-fetch run-to-run variability, while insert variability is large enough on both engines that no fine insert ordering is claimed (Supplement Tables S1, S9), and the ordering holds across the concurrency sweep at ≥ 32 connections (Supplement Figure S2). Rather than assert a specific statistical power, absent a prespecified effect-size model, we rest the conclusions on effect sizes and interval estimates, foremost the *predefined* native-driver contrasts (Supplement Table S41), keep-

ing the ranking-selected adjacent-pair permutation tests only as a descriptive post-hoc check. Application processes do not cross adapter, engine, or replicate blocks, although read endpoints within a block share one process [50]. All blocks share one host; primary intervals remain within-campaign, while the corrected RQ2 subset has a second same-host campaign. The secondary experiments covering only a layer subset or few replicates (Supplement Table S32) are read as *sensitivity evidence* on the tested subset, not as claims about all eleven implementations. The common-SQL contrast and the matched-utilization tails rest on few replicates, so both are reported with their observed spread and neither carries an interval estimate (Supplement Tables S14, S21–S22). The matched-utilization tails carry a second, unpropagated uncertainty: each offered rate is a fraction of an *estimated* capacity, whose spread is evaluated separately (Supplement Table S45) rather than carried into the reported p99. They are secondary evidence for the operating-point distinction, not a precise tail estimate.

7. Conclusion and Future Work

This paper proposes a concrete discipline for access-layer comparisons: derive expected read results from the declared task, check mutual equivalence and post-write state before a timing is admitted, define each configured treatment by a predeclared policy, and report fixed-demand latency separately from capacity and matched utilization. The selection policy is reproducible evidence about the treatments it fixes, not observed practitioner behavior, and the common-SQL raw-path experiment is a compound sensitivity contrast, not a library-intrinsic overhead estimate.

In the pinned Express.js experiment, the largest observed gap occurs on the tested deep fetch: $7.01\times$ on PostgreSQL and $5.12\times$ on MySQL, narrowing, in a separate ten-run campaign, to $1.71\times$ and $2.03\times$ under common SQL through raw facilities. At stable 50% and most 70% capacity targets, p99 values form a much smaller band than at the high-load point. At the stated evidential bar only one treatment reverses durably across the two tested backend stacks, and

it is the one whose MySQL driver differs, so the experiment records limited, treatment-specific non-transfer without identifying the DBMS as its source.

The evidence record separates completed checks from unperformed validation: the oracle, preflight, second same-host campaign, checklist, and retrospective audit make the method inspectable, while the audit has one rater, every adapter is single-author, and both campaigns share one virtualized host. Result-blind packets are provided instead of a claim of independent validation.

Future work should estimate inter-rater agreement for the external audit, repeat the reduced RQ2 matrix on a physically independent host, and extend the workload across datasets, topologies, and cold-cache regimes. The first is prepared rather than proposed: a rater-extensible audit record and a committed agreement scorer ship with the artifact.

CRedit authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The author is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Data availability

The public repository is available at <https://github.com/mmiotk/express-db-access-performance>. The complete revision artifact is archived as release v1.14.0, which the concept DOI <https://doi.org/10.5281/zenodo.21313858> resolves to [51]; the Zenodo record also carries an immutable

per-version DOI for that release. A reproducibility summary — versions, requirements, run commands, resources needed, and which results the harness regenerates automatically from the archived raw data — is in Supplement Table S31 and REPRODUCE.md. **Exact reproduction requires the reference engines** (PostgreSQL 18.4 and MySQL 9.7.1) via the user-space conda path documented there; the convenience Docker path pins the same engine versions but uses relaxed durability in a containerized topology, so it reproduces the workflow, not the headline default-durability insert numbers. A machine-readable encoding of the protocol ships as `protocol-checklist.yaml`. Reproduction status is separated explicitly: author-run isolated reconstructions verified the archived JSON hashes and regenerated the machine-generated tables byte-for-byte without starting either DBMS (dates and counts in Supplement Table S31). Neither is independent reproduction nor a re-execution of the current measurements, and neither is described as such.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) to draft and edit manuscript text and improve its readability and language, and OpenAI Codex twice to obtain independent critical reviews of the complete manuscript. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article. AI assistance that formed part of the research itself is separate from the preparation use declared here and is documented in Section 3.9: it covered implementation of the benchmark harness and analysis code, and extended to

proposing candidate analysis procedures, which the author selected among, verified, and interpreted. That section states the models, the affected components, the checks that independently validate the generated code, and the limits of those checks.

References

- [1] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Detecting performance anti-patterns for applications developed using object-relational mapping, in: Proceedings of the 36th International Conference on Software Engineering (ICSE), ACM, 2014, pp. 1001–1012. doi:10.1145/2568225.2568259.
- [2] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, Boston, MA, 2002.
- [3] I. K. Chaniotis, K.-I. D. Kyriakou, N. D. Tselikas, Is Node.js a viable option for building modern web applications? a performance evaluation study, Computing 97 (10) (2015) 1023–1044. doi:10.1007/s00607-014-0394-9.
- [4] P. Kuffel, B. Walter, Changes in Node.js server-side application performance characteristics over time under load, in: Advances in Information Systems Development, Vol. 77 of Lecture Notes in Information Systems and Organisation, Springer, 2025, pp. 275–294. doi:10.1007/978-3-031-87880-0_14.
- [5] K. X. Carmo, F. Ferreira, E. Figueiredo, Performance evaluation of back-end frameworks: A comparative study, in: Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI), ACM, 2024, pp. 1–9. doi:10.1145/3658271.3658314.
- [6] M. Collina, et al., autocannon: fast http/1.1 benchmarking tool for node.js, <https://github.com/mcollina/autocannon>, version 8.0.0; accessed 10 July 2026. (2023).

- [7] M. Raasveldt, P. Holanda, T. Gubner, H. Mühleisen, Fair benchmarking considered difficult: Common pitfalls in database performance testing, in: *Proceedings of the Workshop on Testing Database Systems (DBTest)*, ACM, 2018, pp. 1–6. doi:10.1145/3209950.3209955.
- [8] M. Fruth, S. Scherzinger, W. Maurer, R. Ramsauer, Tell-tale tail latencies: Pitfalls and perils in database benchmarking, in: *Performance Evaluation and Benchmarking (TPCTC 2021)*, Vol. 13169 of *Lecture Notes in Computer Science*, Springer, 2022, pp. 119–134. doi:10.1007/978-3-030-94437-7_8.
- [9] A. Georges, D. Buytaert, L. Eeckhout, Statistically rigorous Java performance evaluation, in: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, ACM, 2007, pp. 57–76. doi:10.1145/1297027.1297033.
- [10] T. Sotiropoulos, S. Chaliasos, V. Atlidakis, D. Mitropoulos, D. Spinellis, Data-oriented differential testing of object-relational mapping systems, in: *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*, IEEE, 2021, pp. 1535–1547. doi:10.1109/ICSE43902.2021.00137.
- [11] C. Ireland, D. Bowers, M. Newton, K. Waugh, A classification of object-relational impedance mismatch, in: *2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA)*, IEEE, 2009, pp. 36–43. doi:10.1109/DBKDA.2009.11.
- [12] A. Torres, R. Galante, M. S. Pimenta, A. J. B. Martins, Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design, *Inf. Softw. Technol.* 82 (2017) 1–18. doi:10.1016/j.infsof.2016.09.009.
- [13] J. Yang, P. Subramaniam, S. Lu, C. Yan, A. Cheung, How not to structure your database-backed web applications: A study of performance bugs in the

- wild, in: Proceedings of the 40th International Conference on Software Engineering (ICSE), ACM, 2018, pp. 800–810. doi:10.1145/3180155.3180194.
- [14] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational mapping frameworks, *IEEE Trans. Softw. Eng.* 42 (12) (2016) 1148–1161. doi:10.1109/TSE.2016.2553039.
 - [15] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, H. Plattner, Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies, in: Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS), 2017. doi:10.24251/HICSS.2017.592.
 - [16] P. van Zyl, D. G. Kourie, L. Coetzee, A. Boake, The influence of optimisations on the performance of an object relational mapping tool, in: Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT), ACM, 2009, pp. 150–159. doi:10.1145/1632149.1632169.
 - [17] A. Bonvoisin, C. Quinton, R. Rouvoy, Understanding the performance-energy tradeoffs of object-relational mapping frameworks, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2024, pp. 626–636. doi:10.1109/SANER60148.2024.00069.
 - [18] D. Colley, C. Stanier, M. Asaduzzaman, The impact of object-relational mapping frameworks on relational query performance, in: 2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE), IEEE, 2018, pp. 47–52, java, C#, Python, PHP — deliberately excludes JavaScript/Node. doi:10.1109/iccecome.2018.8659222.
 - [19] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, R. R. Siswanto, Optimizing database access strategy: A performance analysis comparison

- of raw sql and prisma orm, *Procedia Comput. Sci.* 269 (2025) 1201–1210. doi:10.1016/j.procs.2025.09.061.
- [20] J. Attala, C. Khemapatpapan, Comparing the performance of ORMs, *J. Comput. Creat. Technol.* 3 (2) (2025) 201–213. doi:10.14456/jcct.2025.16. URL <https://so13.tci-thaijo.org/index.php/jcct/article/view/2330>
- [21] S. Zhadko-Bazilevych, Analysis of ORM framework approaches for Node.js, *J. Comput. Sci. Inst.* 37 (2025) 426–430. doi:10.35784/jcsi.7951.
- [22] I. P. A. E. Pratama, I. M. S. Raharja, Node.js performance benchmarking and analysis at VirtualBox, Docker, and Podman environment using Node-Bench method, *Int. J. Inform. Vis.* 7 (4) (2023) 2240–2246. doi:10.62527/joiv.7.4.1762.
- [23] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, M. A. Ibrahim, Optimizing asynchronous performance in Node.js with Express and PostgreSQL, *Procedia Comput. Sci.* 269 (2025) 172–181. doi:10.1016/j.procs.2025.08.270.
- [24] S. V. Salunke, A. Ouda, A performance benchmark for the PostgreSQL and MySQL databases, *Future Internet* 16 (10) (2024) 382. doi:10.3390/fi16100382.
- [25] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with YCSB, in: *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, ACM, 2010, pp. 143–154. doi:10.1145/1807128.1807152.
- [26] J. Dean, L. A. Barroso, The tail at scale, *Commun. ACM* 56 (2) (2013) 74–80. doi:10.1145/2408776.2408794.
- [27] J. Li, N. K. Sharma, D. R. K. Ports, S. D. Gribble, Tales of the tail: Hardware, OS, and application-level sources of tail latency, in: *Proceedings*

- of the ACM Symposium on Cloud Computing (SoCC), ACM, 2014, pp. 1–14. doi:10.1145/2670979.2670988.
- [28] P. Leitner, C.-P. Bezemer, An exploratory study of the state of practice of performance testing in Java-based open source projects, in: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE), ACM, 2017, pp. 373–384. doi:10.1145/3030207.3030213.
 - [29] Drizzle Team, Drizzle orm benchmarks, <https://orm.drizzle.team/benchmarks>, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026). (2024).
 - [30] Prisma, Query performance benchmarks, <https://benchmarks.prisma.io/>, prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026). (2024).
 - [31] Gel Data (EdgeDB), Imdbench: Orm benchmarks, <https://github.com/geldata/imdbench>, accessed 10 July 2026. (2024).
 - [32] S. Kounev, K.-D. Lange, J. von Kistowski, Systems Benchmarking: For Scientists and Engineers, Springer, 2020. doi:10.1007/978-3-030-41705-5.
 - [33] W. Hasselbring, Benchmarking as empirical standard in software engineering research, in: Proceedings of the Evaluation and Assessment in Software Engineering (EASE), ACM, 2021, pp. 457–462. doi:10.1145/3463274.3463361.
 - [34] M. Rigger, Z. Su, Testing database engines via pivoted query synthesis, in: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), USENIX Association, 2020, pp. 667–682.
 - [35] V. R. Basili, H. D. Rombach, The TAME project: Towards improvement-oriented software environments, IEEE Trans. Softw. Eng. 14 (6) (1988) 758–773. doi:10.1109/32.6156.

- [36] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, *Experimentation in Software Engineering*, Springer, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29044-2.
- [37] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, *IEEE Trans. Softw. Eng.* 28 (8) (2002) 721–734. doi:10.1109/TSE.2002.1027796.
- [38] S. Harizopoulos, D. J. Abadi, S. Madden, M. Stonebraker, OLTP through the looking glass, and what we found there, in: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, ACM, 2008, pp. 981–992. doi:10.1145/1376616.1376713.
- [39] K. Gupta, M. Mathuria, Improving performance of web application approaches using connection pooling, in: *2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA)*, IEEE, 2017, pp. 355–358. doi:10.1109/ICECA.2017.8212833.
- [40] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, M. Kalinowski, Data management in microservices: State of the practice, challenges, and research directions, *Proc. VLDB Endow.* 14 (13) (2021) 3348–3361. doi:10.14778/3484224.3484232.
- [41] B. Schroeder, A. Wierman, M. Harchol-Balter, Open versus closed: A cautionary tale, in: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2006. URL <https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html>
- [42] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, M. Stonebraker, Staring into the abyss: An evaluation of concurrency control with one thousand cores, *Proc. VLDB Endow.* 8 (3) (2014) 209–220. doi:10.14778/2735508.2735511.

- [43] G. Tene, How NOT to measure latency, <https://www.infoq.com/presentations/latency-response-time/>, conference presentation, QCon San Francisco 2015 (accessed 10 July 2026). (2015).
- [44] B. Krishnamachari, How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results, <https://arxiv.org/abs/2605.00428> (2026).
- [45] C. Laaber, J. Scheuner, P. Leitner, Software microbenchmarking in the cloud. how bad is it really?, *Empir. Softw. Eng.* 24 (4) (2019) 2469–2508. doi:10.1007/s10664-019-09681-1.
- [46] A. Arcuri, L. Briand, A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering, *Softw. Test. Verif. Reliab.* 24 (3) (2014) 219–250. doi:10.1002/stvr.1486.
- [47] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, *Psychol. Bull.* 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.
- [48] T. D. Cook, D. T. Campbell, *Quasi-Experimentation: Design and Analysis Issues for Field Settings*, Houghton Mifflin, Boston, MA, 1979.
- [49] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.
- [50] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, ACM, 2013, pp. 63–74. doi:10.1145/2464157.2464160.
- [51] M. Miotk, Replication package for A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js, Zenodo, version 1.14.0, [software and dataset] (2026). doi:10.5281/zenodo.21313858.